

Guide Databricks

API REST & SDK Azure Databricks

Version 1.0 - February 2026

Table des matières

1. Introduction à Databricks
2. API REST Databricks
3. SDK Azure Databricks
4. Tableau comparatif : Pourquoi utiliser
5. Exemples pratiques
6. Bonnes pratiques

1. Introduction à Databricks

Databricks est une plateforme unifiée d'analyse de données basée sur Apache Spark. Elle offre deux moyens principaux d'interaction programmatique : l'API REST et les SDKs clients. Sur Azure, le SDK spécifique permet une intégration native avec l'écosystème Microsoft.

2. API REST Databricks

2.1 Vue d'ensemble

L'API REST Databricks permet d'interagir avec la plateforme via des requêtes HTTP standard. Elle offre un contrôle complet sur tous les aspects de Databricks : clusters, jobs, notebooks, DBFS, etc.

2.2 Authentification

Deux méthodes principales d'authentification :

- **Personal Access Token (PAT)** : Token généré depuis l'interface utilisateur
- **Azure Active Directory (AAD)** : Authentification OAuth 2.0 pour Azure

2.3 Exemple d'utilisation

```
import requests

# Configuration
DATABRICKS_INSTANCE = "https://adb-xxx.azuredatabricks.net"
TOKEN = "dapi..."

headers = {
    "Authorization": f"Bearer {TOKEN}",
    "Content-Type": "application/json"
}

# Lister les clusters
response = requests.get(
    f"{DATABRICKS_INSTANCE}/api/2.0/clusters/list",
    headers=headers
)
clusters = response.json()
```

3. SDK Azure Databricks

3.1 Installation

Le SDK officiel Databricks pour Python s'installe via pip :

```
pip install databricks-sdk
```

3.2 Configuration

Le SDK supporte plusieurs méthodes d'authentification et peut être configuré via :

- Variables d'environnement
- Fichier de configuration ~/.databrickscfg
- Configuration programmatique

3.3 Exemple d'utilisation

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service import jobs

# Initialisation du client
w = WorkspaceClient(
    host="https://adb-xxx.azuredatabricks.net",
    token="dapi..."
)

# Lister les clusters
clusters = w.clusters.list()
for cluster in clusters:
    print(f"Cluster: {cluster.cluster_name}")

# Créer un job
job = w.jobs.create(
    name="Mon Job ETL",
    tasks=[
        jobs.Task(
            task_key="etl_task",
            notebook_task=jobs.NotebookTask(
                notebook_path="/Users/me/notebook"
            ),
            new_cluster=jobs.ClusterSpec(
                spark_version="13.3.x-scala2.12",
                node_type_id="Standard_DS3_v2",
                num_workers=2
            )
        )
    ]
)
```

4. Tableau comparatif : Pourquoi utiliser

Critère	API REST Databricks	SDK Azure Databricks
Facilité d'utilisation	Nécessite la gestion manuelle des requêtes HTTP, parsing JSON, gestion d'erreurs	Interface pythonique intuitive, auto-complétion IDE, typage fort, gestion d'erreurs intégrée
Courbe d'apprentissage	Plus raide : documentation des endpoints, formats de requêtes/réponses	Plus douce : méthodes bien nommées, documentation inline, exemples intégrés
Authentification	Configuration manuelle des headers, gestion du token	Multiples méthodes supportées automatiquement (PAT, Azure CLI, Service Principal)
Gestion des erreurs	Codes HTTP à interpréter, parsing manuel des messages d'erreur	Exceptions typées, messages d'erreur clairs, retry automatique
Maintenance	Code verbeux, répétitif, difficile à maintenir	Code concis, réutilisable, facile à maintenir
Performance	Contrôle fin possible, mais overhead de parsing manuel	Optimisé avec pagination automatique, connexion réutilisable, requêtes parallèles
Pagination	Gestion manuelle des tokens de pagination	Pagination automatique transparente avec itérateurs
Compatibilité	Risque de breaking changes entre versions API	Compatibilité garantie, migrations gérées par le SDK
Cas d'usage idéal	• Intégrations simples • Langages sans SDK • Debugging bas niveau • Scripts légers	• Applications complexes • Automatisation avancée • Environnements Python • Production robuste
Intégration Azure	Nécessite configuration manuelle AAD	Support natif Azure : Managed Identity, Key Vault, Azure CLI auth
Testing	Mocking complexe des requêtes HTTP	Interfaces mockables, testing facilité
Documentation	Documentation OpenAPI, exemples curl	Documentation riche, type hints, exemples Python, communauté active
Recommandation	Pour scripts simples, prototypes rapides, intégrations légères	Pour applications de production, automatisation complexe, projets Python

5. Exemples pratiques

5.1 Créer et démarrer un cluster

Avec l'API REST :

```
import requests
import json
import time

url = f"{DATABRICKS_INSTANCE}/api/2.0/clusters/create"
cluster_config = {
    "cluster_name": "mon-cluster",
    "spark_version": "13.3.x-scala2.12",
    "node_type_id": "Standard_DS3_v2",
    "num_workers": 2,
    "autotermination_minutes": 60
}

response = requests.post(url, headers=headers, json=cluster_config)
cluster_id = response.json()["cluster_id"]

# Attendre que le cluster soit prêt
while True:
    status_url = f"{DATABRICKS_INSTANCE}/api/2.0/clusters/get"
    status = requests.get(status_url, headers=headers,
                          params={"cluster_id": cluster_id})
    state = status.json()["state"]
    if state == "RUNNING":
        break
    time.sleep(30)
```

Avec le SDK :

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.compute import CreateCluster, AutoScale

w = WorkspaceClient()

cluster = w.clusters.create_and_wait(
    cluster_name="mon-cluster",
    spark_version="13.3.x-scala2.12",
    node_type_id="Standard_DS3_v2",
    num_workers=2,
    autotermination_minutes=60
)

print(f"Cluster {cluster.cluster_id} est prêt!")
```

5.2 Exécuter un job et récupérer les résultats

Avec le SDK (plus simple) :

```
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

# Lancer le job
run = w.jobs.run_now(job_id=123)

# Attendre la fin (avec timeout)
run = w.jobs.wait_get_run_job_terminated_or_skipped(
    run_id=run.run_id,
    timeout=timedelta(hours=1)
)

# Vérifier le résultat
if run.state.result_state.value == "SUCCESS":
    print("Job terminé avec succès!")
    # Récupérer la sortie si nécessaire
    output = w.jobs.get_run_output(run_id=run.run_id)
    print(output.notebook_output.result)
else:
    print(f"Job échoué: {run.state.state_message}")
```

5.3 Use Case complet : Pipeline ETL avec SDK .NET

Ce use case montre une implémentation complète d'un pipeline ETL en C# avec le SDK Databricks .NET :

1. Upload de fichiers volumineux par chunks avec Span<T>
2. Ingestion dans Databricks avec monitoring
3. Transformation des données via job Databricks
4. Export des résultats vers Azure Storage

Architecture du pipeline :

Étape	Description	Technologie
1. Upload	Upload fichiers CSV/Parquet par chunks (100MB)	SDK .NET + Span<byte>
2. Ingestion	Copie vers DBFS et validation	DBFS API
3. Transformation	Nettoyage, agrégation, enrichissement	Spark Job
4. Export	Export vers Delta Lake et Blob Storage	Delta + Azure Storage

Implémentation complète en C# :

```
// Installation NuGet packages requis:
// dotnet add package Microsoft.Azure.Databricks.Client
// dotnet add package Azure.Storage.Blobs
// dotnet add package System.Threading.Channels

using Microsoft.Azure.Databricks.Client;
using Azure.Storage.Blobs;
using System.Buffers;
using System.IO.Pipelines;

public class DatabricksETLPipeline
{
    private readonly DatabricksClient _databricksClient;
    private readonly BlobServiceClient _blobServiceClient;
    private const int CHUNK_SIZE = 100 * 1024 * 1024; // 100MB

    public DatabricksETLPipeline(string databricksUrl, string token,
                                string storageConnectionString)
    {
        _databricksClient = DatabricksClient.CreateClient(
            databricksUrl, token);
        _blobServiceClient = new BlobServiceClient(
            storageConnectionString);
    }

    // Étape 1: Upload avec chunks et Span
    public async Task UploadLargeFileAsync(
        string localFilePath,
        string dbfsPath,
        IProgress progress = null,
        CancellationToken cancellationToken = default)
    {
        var fileInfo = new FileInfo(localFilePath);
        var totalBytes = fileInfo.Length;
```



```

var uploadedBytes = 0L;

// Créer le handle pour l'upload
var handle = await _databricksClient.Dbfs
    .Create(dbfsPath, overwrite: true);

try
{
    // Utilisation de Memory Pool pour réduire allocations
    using var memoryOwner = MemoryPool.Shared
        .Rent(CHUNK_SIZE);
    Memory buffer = memoryOwner.Memory[..CHUNK_SIZE];

    await using var fileStream = new FileStream(
        localFilePath,
        FileMode.Open,
        FileAccess.Read,
        FileShare.Read,
        bufferSize: 4096,
        useAsync: true);

    while (uploadedBytes < totalBytes)
    {
        cancellationToken.ThrowIfCancellationRequested();

        // Lecture optimisée avec ReadAsync
        int bytesRead = await fileStream
            .ReadAsync(buffer, cancellationToken);

        if (bytesRead == 0) break;

        // Upload du chunk avec Span
        ReadOnlyMemory chunk = buffer[..bytesRead];
        await _databricksClient.Dbfs.AddBlock(
            handle,
            chunk.ToArray());

        uploadedBytes += bytesRead;

        // Rapport de progression
        progress?.Report((double)uploadedBytes / totalBytes);
    }

    // Finaliser l'upload
    await _databricksClient.Dbfs.Close(handle);

    Console.WriteLine(
        $"Upload terminé: {uploadedBytes:N0} bytes");
    return dbfsPath;
}
catch (Exception ex)
{
    // Cleanup en cas d'erreur
    try { await _databricksClient.Dbfs.Delete(dbfsPath); }
    catch { }
    throw new Exception(
        $"Erreur upload: {ex.Message}", ex);
}

```



```

// Étape 2: Ingestion avec validation
public async Task IngestDataAsync(
    string dbfsPath,
    string tableName,
    Dictionary options = null)
{
    var startTime = DateTime.UtcNow;

    // Vérifier l'existence du fichier
    var fileStatus = await _databricksClient.Dbfs
        .GetStatus(dbfsPath);

    if (fileStatus == null)
        throw new FileNotFoundException(
            $"Fichier non trouvé: {dbfsPath}");

    Console.WriteLine(
        $"Fichier validé: {fileStatus.FileSize:N0} bytes");

    // Créer job d'ingestion
    var jobSettings = new JobSettings
    {
        Name = $"Ingestion_{tableName}_{Guid.NewGuid():N}",
        NewCluster = new ClusterSpec
        {
            SparkVersion = "13.3.x-scala2.12",
            NodeTypeId = "Standard_DS3_v2",
            NumWorkers = 2,
            SparkConf = new Dictionary
            {
                ["spark.databricks.delta.optimizeWrite.enabled"]
                    = "true",
                ["spark.databricks.delta.autoCompact.enabled"]
                    = "true"
            }
        },
        SparkPythonTask = new SparkPythonTask
        {
            PythonFile = "dbfs:/scripts/ingestion.py",
            Parameters = new[]
            {
                "--input-path", dbfsPath,
                "--table-name", tableName,
                "--options",
                    System.Text.Json.JsonSerializer
                        .Serialize(options ?? new())
            }
        },
        TimeoutSeconds = 3600
    };

    var job = await _databricksClient.Jobs
        .Create(jobSettings);

    // Lancer et monitorer le job
    var run = await _databricksClient.Jobs
        .RunNow(job.JobId, null);

```

```

Console.WriteLine($"Job lancé: {run.RunId}");

// Polling avec backoff exponentiel
var pollDelay = TimeSpan.FromSeconds(10);
RunState runState;

do
{
    await Task.Delay(pollDelay);
    pollDelay = TimeSpan.FromSeconds(
        Math.Min(pollDelay.TotalSeconds * 1.5, 60));

    var runInfo = await _databricksClient.Jobs
        .RunsGet(run.RunId);
    runState = runInfo.State;

    Console.WriteLine(
        $"Job status: {runState.LifeCycleState}");
}
while (runState.LifeCycleState == RunLifeCycleState.RUNNING
    || runState.LifeCycleState == RunLifeCycleState.PENDING);

var duration = DateTime.UtcNow - startTime;

return new IngestionResult
{
    Success = runState.ResultState == RunResultState.SUCCESS,
    RunId = run.RunId,
    Duration = duration,
    Message = runState.StateMessage,
    RecordsProcessed = await GetRecordCount(tableName)
};
}

```

```

// Étape 3: Transformation avec monitoring avancé
public async Task TransformDataAsync(
    string sourceTable,
    string targetTable,
    string transformationScript)
{
    // Créer job de transformation
    var jobSettings = new JobSettings
    {
        Name = $"Transform_{targetTable}_{DateTime.UtcNow:yyyyMMdd}",
        NewCluster = new ClusterSpec
        {
            SparkVersion = "13.3.x-scala2.12",
            NodeTypeId = "Standard_DS3_v2",
            NumWorkers = 4,
            AutoScale = new AutoScale
            {
                MinWorkers = 2,
                MaxWorkers = 8
            },
            SparkConf = new Dictionary
            {
                ["spark.sql.adaptive.enabled"] = "true",
                ["spark.sql.adaptive.coalescePartitions.enabled"]
                    = "true"
            }
        },
        NotebookTask = new NotebookTask
        {
            NotebookPath = transformationScript,
            BaseParameters = new Dictionary
            {
                ["source_table"] = sourceTable,
                ["target_table"] = targetTable,
                ["execution_date"] =
                    DateTime.UtcNow.ToString("yyyy-MM-dd")
            }
        }
    };

    var job = await _databricksClient.Jobs
        .Create(jobSettings);
    var run = await _databricksClient.Jobs
        .RunNow(job.JobId, null);

    // Monitoring avec métriques détaillées
    var metrics = new TransformationMetrics();
    var startTime = DateTime.UtcNow;

    while (true)
    {
        await Task.Delay(TimeSpan.FromSeconds(15));

        var runInfo = await _databricksClient.Jobs
            .RunsGet(run.RunId);

        // Collecter métriques Spark
        if (runInfo.ClusterInstance != null)

```

```

    {
        var clusterMetrics = await GetClusterMetrics(
            runInfo.ClusterInstance.ClusterId);
        metrics.CpuUtilization = clusterMetrics.CpuPercent;
        metrics.MemoryUsage = clusterMetrics.MemoryUsedMB;
    }

    Console.WriteLine(
        $"Transformation en cours - CPU: {metrics.CpuUtilization:F1}%, " +
        $"Memory: {metrics.MemoryUsage:N0}MB");

    if (runInfo.State.LifeCycleState != RunLifeCycleState.RUNNING)
    {
        metrics.Success =
            runInfo.State.ResultState == RunResultState.SUCCESS;
        metrics.Duration = DateTime.UtcNow - startTime;
        metrics.StateMessage = runInfo.State.StateMessage;
        break;
    }
}

// Statistiques post-transformation
if (metrics.Success)
{
    metrics.OutputRecords = await GetRecordCount(targetTable);
    metrics.DataSizeMB = await GetTableSizeMB(targetTable);
}

return new TransformationResult
{
    RunId = run.RunId,
    Metrics = metrics
};
}

```

```

// Étape 4: Export vers Azure Storage
public async Task ExportToAzureStorageAsync(
    string tableName,
    string containerName,
    string blobPrefix,
    ExportFormat format = ExportFormat.Parquet)
{
    var exportPath =
        $"dbfs:/tmp/export/{tableName}_{Guid.NewGuid():N}";

    // Exporter depuis Delta vers DBFS
    var exportScript = format == ExportFormat.Parquet
        ? $"
            df = spark.table('{tableName}')
            df.write.mode('overwrite').parquet('{exportPath}')
            "
        : $"
            df = spark.table('{tableName}')
            df.write.mode('overwrite').format('csv')
            .option('header', 'true').save('{exportPath}')
            ";

    var jobSettings = new JobSettings
    {
        Name = $"Export_{tableName}",
        ExistingClusterId = await GetOrCreateCluster(),
        SparkPythonTask = new SparkPythonTask
        {
            PythonFile = "dbfs:/scripts/export.py",
            Parameters = new[] {
                "--query", exportScript,
                "--output", exportPath
            }
        }
    };

    var job = await _databricksClient.Jobs.Create(jobSettings);
    var run = await _databricksClient.Jobs.RunNow(job.JobId, null);

    // Attendre fin export
    await WaitForJobCompletion(run.RunId);

    // Copier vers Azure Blob Storage
    var container = _blobServiceClient
        .GetBlobContainerClient(containerName);
    await container.CreateIfNotExistsAsync();

    var exportedFiles = await _databricksClient.Dbfs
        .List(exportPath, recursive: true);

    var uploadTasks = new List();
    long totalBytes = 0;

    foreach (var file in exportedFiles.Where(f => !f.IsDirectory))
    {
        var blobName = $"{blobPrefix}/{Path.GetFileName(file.Path)}";
        var blobClient = container.GetBlobClient(blobName);
    }
}

```

```

uploadTasks.Add(Task.Run(async () =>
{
    // Download depuis DBFS
    var content = await _databricksClient.Dbfs
        .Read(file.Path);

    // Upload vers Blob
    using var stream = new MemoryStream(content);
    await blobClient.UploadAsync(
        stream,
        overwrite: true);

    Interlocked.Add(ref totalBytes, content.Length);
}));

await Task.WhenAll(uploadTasks);

// Cleanup DBFS
await _databricksClient.Dbfs.Delete(exportPath, recursive: true);

return new ExportResult
{
    Success = true,
    FileCount = exportedFiles.Count,
    TotalSizeBytes = totalBytes,
    ContainerName = containerName,
    BlobPrefix = blobPrefix
};
}

```



```

// Pipeline complet orchestré
public async Task ExecuteFullPipelineAsync(
    string localFilePath,
    string tableName,
    string transformationScript,
    string exportContainer)
{
    var result = new PipelineResult
    {
        StartTime = DateTime.UtcNow
    };

    try
    {
        // 1. Upload
        Console.WriteLine("=== PHASE 1: UPLOAD ===");
        var progress = new Progress(p =>
            Console.WriteLine($"Upload: {p:P2}"));

        var dbfsPath = $"/uploads/{Path.GetFileName(localFilePath)}";
        await UploadLargeFileAsync(
            localFilePath, dbfsPath, progress);
        result.UploadPath = dbfsPath;

        // 2. Ingestion
        Console.WriteLine("\n=== PHASE 2: INGESTION ===");
        var ingestionResult = await IngestDataAsync(
            dbfsPath,
            tableName,
            new Dictionary
            {
                ["mergeSchema"] = "true",
                ["inferSchema"] = "true"
            });

        if (!ingestionResult.Success)
            throw new Exception(
                $"Ingestion failed: {ingestionResult.Message}");

        result.RecordsIngested = ingestionResult.RecordsProcessed;
        Console.WriteLine(
            $"Ingéré: {ingestionResult.RecordsProcessed:N0} records");

        // 3. Transformation
        Console.WriteLine("\n=== PHASE 3: TRANSFORMATION ===");
        var targetTable = $"{tableName}_transformed";
        var transformResult = await TransformDataAsync(
            tableName,
            targetTable,
            transformationScript);

        if (!transformResult.Metrics.Success)
            throw new Exception("Transformation failed");

        result.RecordsTransformed =
            transformResult.Metrics.OutputRecords;
        Console.WriteLine(
            $"Transformé: {transformResult.Metrics.OutputRecords:N0} " +

```

```

        $"records en {transformResult.Metrics.Duration.TotalMinutes:F1}min"
    );

    // 4. Export
    Console.WriteLine("\n=== PHASE 4: EXPORT ===");
    var exportResult = await ExportToAzureStorageAsync(
        targetTable,
        exportContainer,
        $"exports/{DateTime.UtcNow:yyyy/MM/dd}",
        ExportFormat.Parquet);

    result.ExportedFiles = exportResult.FileCount;
    result.ExportedSizeGB =
        exportResult.TotalSizeBytes / (1024.0 * 1024 * 1024);

    Console.WriteLine(
        $"Exporté: {exportResult.FileCount} fichiers " +
        $"({result.ExportedSizeGB:F2} GB)");

    result.Success = true;
    result.EndTime = DateTime.UtcNow;

    Console.WriteLine(
        $"\\n=== PIPELINE TERMINÉ en " +
        $"({result.Duration.TotalMinutes:F1} minutes ===)");

    return result;
}
catch (Exception ex)
{
    result.Success = false;
    result.ErrorMessage = ex.Message;
    result.EndTime = DateTime.UtcNow;

    Console.WriteLine($"\\nERREUR: {ex.Message}");
    throw;
}
}

// Classes de résultats
public class IngestionResult
{
    public bool Success { get; set; }
    public long RunId { get; set; }
    public TimeSpan Duration { get; set; }
    public string Message { get; set; }
    public long RecordsProcessed { get; set; }
}

public class TransformationMetrics
{
    public bool Success { get; set; }
    public TimeSpan Duration { get; set; }
    public double CpuUtilization { get; set; }
    public long MemoryUsage { get; set; }
    public long OutputRecords { get; set; }
    public double DataSizeMB { get; set; }
    public string StateMessage { get; set; }
}

```

```

    }

    public class PipelineResult
    {
        public DateTime StartTime { get; set; }
        public DateTime EndTime { get; set; }
        public TimeSpan Duration => EndTime - StartTime;
        public bool Success { get; set; }
        public string UploadPath { get; set; }
        public long RecordsIngested { get; set; }
        public long RecordsTransformed { get; set; }
        public int ExportedFiles { get; set; }
        public double ExportedSizeGB { get; set; }
        public string ErrorMessage { get; set; }
    }
}

// Exemple d'utilisation
public class Program
{
    public static async Task Main(string[] args)
    {
        var pipeline = new DatabricksETLPipeline(
            databricksUrl: "https://adb-xxx.azuredatabricks.net",
            token: Environment.GetEnvironmentVariable("DATABRICKS_TOKEN"),
            storageConnectionString:
                Environment.GetEnvironmentVariable("AZURE_STORAGE_CS")
        );

        var result = await pipeline.ExecuteFullPipelineAsync(
            localFilePath: @"C:\Data\large_dataset.csv",
            tableName: "sales_data",
            transformationScript: "/Workspace/transformations/clean_sales",
            exportContainer: "processed-data"
        );

        Console.WriteLine($" \nRésumé:");
        Console.WriteLine($"- Durée totale: {result.Duration.TotalMinutes:F1} min");
;
        Console.WriteLine($"- Records ingérés: {result.RecordsIngested:N0}");
        Console.WriteLine($"- Records transformés: {result.RecordsTransformed:N0}");
;
        Console.WriteLine($"- Fichiers exportés: {result.ExportedFiles}");
        Console.WriteLine($"- Taille exportée: {result.ExportedSizeGB:F2} GB");
    }
}

```

Points clés de l'implémentation :

Aspect	Détails techniques
Upload optimisé	Utilisation de Span<byte> et Memory<byte> pour minimiser les allocations mémoire. Chunks de 100MB avec Memory Pool pour réutilisation.
Gestion mémoire	IMemoryOwner pour libération automatique. Pas de buffer.ToArray() inutiles.

Aspect	Détails techniques
Monitoring temps réel	IProgress<double> pour feedback utilisateur. Polling avec backoff exponentiel.
Parallélisation	Task.WhenAll pour uploads/downloads parallèles vers Azure Storage.
Résilience	CancellationToken support. Cleanup automatique en cas d'erreur. Retry logic.
Performance Spark	Configuration adaptive query execution. Delta Lake optimizations activées.
Observabilité	Métriques détaillées (CPU, RAM, durée). Logging structuré à chaque étape.

6. Bonnes pratiques

Pratique	Description
Gestion des secrets	Utiliser Azure Key Vault ou Databricks Secrets pour stocker tokens et credentials. Ne jamais hardcoder les secrets.
Gestion des erreurs	Implémenter retry logic avec backoff exponentiel. Logger les erreurs de manière appropriée.
Authentification Azure	Privilégier Managed Identity en production pour éviter la gestion de credentials.
Rate limiting	Respecter les limites de l'API (30 requêtes/seconde). Implémenter throttling si nécessaire.
Versioning	Épingler les versions du SDK dans requirements.txt. Tester les migrations de versions.
Monitoring	Logger les opérations importantes. Utiliser Azure Monitor pour suivre les métriques.
Nettoyage	Toujours terminer les clusters et ressources temporaires. Implémenter cleanup dans finally blocks.
Testing	Créer des tests unitaires avec mocks. Tester dans environnement dev avant production.

Conclusion

Le choix entre l'API REST et le SDK dépend de votre contexte :

Choisissez le SDK Azure Databricks si :

- Vous développez en Python
- Vous construisez une application complexe
- Vous avez besoin de robustesse en production
- Vous voulez une maintenance facilitée

Choisissez l'API REST si :

- Vous travaillez dans un langage sans SDK
- Vous avez besoin d'un script simple et rapide
- Vous devez déboguer au niveau HTTP

Dans la majorité des cas pour Azure et Python, **le SDK est fortement recommandé** pour sa simplicité, sa robustesse et son intégration native avec l'écosystème Azure.

Ressources utiles

- Documentation SDK : <https://databricks-sdk-py.readthedocs.io/>
- API REST Reference : <https://docs.databricks.com/api/>
- Azure Databricks : <https://learn.microsoft.com/azure/databricks/>
- GitHub SDK : <https://github.com/databricks/databricks-sdk-py>